

ParcelPilot API and Service Behavior Reference

Endpoint behavior, error handling, request paths, and route isolation.

ParcelPilot operations reference

Contents

1. API conventions
2. POST /login
3. GET /orders/{order_id}
4. GET /orders/{order_id}/tracking
5. GET /orders/{order_id}/documents
6. GET /orders/{order_id}/documents/{document_id}
7. POST /notifications/test
8. GET /reports/volume
9. GET /services
10. GET /health
11. Error response shape
12. Route isolation

1. API conventions

Requests use JSON unless the route returns a file. Responses include status and may include request_id. The API service delegates domain work to downstream services.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture request_id when present.
- Check the downstream service used by the route.

Example

For api conventions, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Normal operating pattern

A normal API request has a request_id, a route, a downstream call when needed, and a final status. Basic health normally completes in tens of milliseconds, while an order route includes the time spent waiting for the orders service.

Failure pattern

A generic API-500 is most useful when correlated with a downstream code on the same request_id. If the downstream service returns a specific error, keep that code visible in the incident record instead of treating API-500 as the final diagnosis.

Worked example

For req-0048123, API-500 at 14:06:18 followed an ORD-500 from the orders service under the same request identifier. That pattern points to the orders path rather than the API process itself.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

2. POST /login

Accepts username and password. Returns a session token on success, 401 for invalid credentials, 423 for locked accounts, and service errors when auth is unavailable.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture request_id when present.
- Check the downstream service used by the route.

Example

For post /login, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Request example

A route investigation should capture method, path, status, application error code, request_id, and a small response excerpt where safe. Comparing the failing request with one known-good request often shows whether the problem is data-specific or service-wide.

If a route calls several services, record which stage failed. This prevents a browser-level message such as 'order page failed' from hiding the difference between order details, tracking, and documents.

Boundary cases

- Expected 404 should remain a controlled not-found response.
- Validation errors should not be rewritten as server failures.
- Partial tracking should stay marked partial.
- A successful enqueue is not proof of immediate notification delivery.

Endpoint documentation describes expected behavior. Current logs and snapshots are still required to diagnose why a particular request differed from that behavior.

3. GET /orders/{order_id}

Reads order details from the orders service. Missing orders return ORD-404. Database or schema problems surface as server errors with ORD codes.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture request_id when present.
- Check the downstream service used by the route.

Example

For get /orders/{order_id}, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Normal operating pattern

Normal order lookup reads the configured SQLite file, finds the requested order, and returns order details independently of tracking. ORD-404 for an unknown identifier is expected application behavior.

Failure pattern

ORD-500 DB_OPEN_FAILED points to the database path or file access. ORD-409 points to schema version mismatch. These failures can both become HTTP 500 responses at the API layer, but the corrective checks are different.

Worked example

If ORDERS_DB_PATH is missing and every tested order ID returns ORD-500, the missing database path is stronger evidence than a simultaneous tracking warning on another request.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

4. GET /orders/{order_id}/tracking

Confirms the order and then retrieves carrier events. It may return partial data or stale-data information without inventing missing events.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture request_id when present.
- Check the downstream service used by the route.

Example

For get /orders/{order_id}/tracking, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Normal operating pattern

Normal order lookup reads the configured SQLite file, finds the requested order, and returns order details independently of tracking. ORD-404 for an unknown identifier is expected application behavior.

Failure pattern

ORD-500 DB_OPEN_FAILED points to the database path or file access. ORD-409 points to schema version mismatch. These failures can both become HTTP 500 responses at the API layer, but the corrective checks are different.

Worked example

If ORDERS_DB_PATH is missing and every tested order ID returns ORD-500, the missing database path is stronger evidence than a simultaneous tracking warning on another request.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

5. GET /orders/{order_id}/documents

Lists documents for an order. Missing optional files should not make the order record unavailable.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture request_id when present.
- Check the downstream service used by the route.

Example

For get /orders/{order_id}/documents, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Normal operating pattern

Normal order lookup reads the configured SQLite file, finds the requested order, and returns order details independently of tracking. ORD-404 for an unknown identifier is expected application behavior.

Failure pattern

ORD-500 DB_OPEN_FAILED points to the database path or file access. ORD-409 points to schema version mismatch. These failures can both become HTTP 500 responses at the API layer, but the corrective checks are different.

Worked example

If ORDERS_DB_PATH is missing and every tested order ID returns ORD-500, the missing database path is stronger evidence than a simultaneous tracking warning on another request.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

6. GET /orders/{order_id}/documents/{document_id}

Returns the file when available. Missing, unreadable, and invalid-store conditions have different DOC codes.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture request_id when present.
- Check the downstream service used by the route.

Example

For get /orders/{order_id}/documents/{document_id}, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Normal operating pattern

Normal order lookup reads the configured SQLite file, finds the requested order, and returns order details independently of tracking. ORD-404 for an unknown identifier is expected application behavior.

Failure pattern

ORD-500 DB_OPEN_FAILED points to the database path or file access. ORD-409 points to schema version mismatch. These failures can both become HTTP 500 responses at the API layer, but the corrective checks are different.

Worked example

If ORDERS_DB_PATH is missing and every tested order ID returns ORD-500, the missing database path is stronger evidence than a simultaneous tracking warning on another request.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

7. POST /notifications/test

Validates a small payload and enqueues a test message. Enqueue success is separate from delivery time.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture request_id when present.
- Check the downstream service used by the route.

Example

For post /notifications/test, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Request example

A route investigation should capture method, path, status, application error code, request_id, and a small response excerpt where safe. Comparing the failing request with one known-good request often shows whether the problem is data-specific or service-wide.

If a route calls several services, record which stage failed. This prevents a browser-level message such as 'order page failed' from hiding the difference between order details, tracking, and documents.

Boundary cases

- Expected 404 should remain a controlled not-found response.
- Validation errors should not be rewritten as server failures.
- Partial tracking should stay marked partial.
- A successful enqueue is not proof of immediate notification delivery.

Endpoint documentation describes expected behavior. Current logs and snapshots are still required to diagnose why a particular request differed from that behavior.

8. GET /reports/volume

Returns grouped operational counts for a valid date range. Large ranges may be slower and ranges over the configured limit are rejected.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture request_id when present.
- Check the downstream service used by the route.

Example

For get /reports/volume, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Normal operating pattern

Reporting reads a separate analytics database. Small date ranges normally complete faster than large ones, and the same report can return different row counts as the requested window changes.

Failure pattern

RPT-422 is a validation problem, RPT-500 is a query failure, and RPT-504 is a timeout after execution begins. A timeout should be investigated with date range, query type, and repeated duration.

Worked example

A 90-day report timing out while 7-day reports complete in 300 ms is not the same as every report failing immediately with RPT-500.

Useful evidence to keep

- The exact incident time or best available time window.
- The route, service, and request_id when available.
- The specific error code rather than only the HTTP status.
- One known-good comparison case when practical.

The operator should also note what was checked and found to be normal. This prevents the next person from repeating the same work and helps keep an unrelated warning from becoming the assumed cause.

9. GET /services

Returns point-in-time service status and latency observations for training investigation.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture request_id when present.
- Check the downstream service used by the route.

Example

For get /services, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Request example

A route investigation should capture method, path, status, application error code, request_id, and a small response excerpt where safe. Comparing the failing request with one known-good request often shows whether the problem is data-specific or service-wide.

If a route calls several services, record which stage failed. This prevents a browser-level message such as 'order page failed' from hiding the difference between order details, tracking, and documents.

Boundary cases

- Expected 404 should remain a controlled not-found response.
- Validation errors should not be rewritten as server failures.
- Partial tracking should stay marked partial.
- A successful enqueue is not proof of immediate notification delivery.

Endpoint documentation describes expected behavior. Current logs and snapshots are still required to diagnose why a particular request differed from that behavior.

10. GET /health

Confirms the API process is running. It does not deeply test every dependency.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture request_id when present.
- Check the downstream service used by the route.

Example

For get /health, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Request example

A route investigation should capture method, path, status, application error code, request_id, and a small response excerpt where safe. Comparing the failing request with one known-good request often shows whether the problem is data-specific or service-wide.

If a route calls several services, record which stage failed. This prevents a browser-level message such as 'order page failed' from hiding the difference between order details, tracking, and documents.

Boundary cases

- Expected 404 should remain a controlled not-found response.
- Validation errors should not be rewritten as server failures.
- Partial tracking should stay marked partial.
- A successful enqueue is not proof of immediate notification delivery.

Endpoint documentation describes expected behavior. Current logs and snapshots are still required to diagnose why a particular request differed from that behavior.

11. Error response shape

Error responses can include `error_code`, `message`, `request_id`, and safe diagnostic details. Specific downstream codes should be preserved when known.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture `request_id` when present.
- Check the downstream service used by the route.

Example

For error response shape, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Request example

A route investigation should capture method, path, status, application error code, `request_id`, and a small response excerpt where safe. Comparing the failing request with one known-good request often shows whether the problem is data-specific or service-wide.

If a route calls several services, record which stage failed. This prevents a browser-level message such as 'order page failed' from hiding the difference between order details, tracking, and documents.

Boundary cases

- Expected 404 should remain a controlled not-found response.
- Validation errors should not be rewritten as server failures.
- Partial tracking should stay marked partial.
- A successful enqueue is not proof of immediate notification delivery.

Endpoint documentation describes expected behavior. Current logs and snapshots are still required to diagnose why a particular request differed from that behavior.

12. Route isolation

A browser page may call several routes. Determine whether order details, tracking, documents, notifications, or reports actually failed before choosing a service.

Expected behavior

The route should return a controlled result that matches the domain condition. Expected not-found or validation behavior should not be converted into a generic server failure.

Troubleshooting

- Capture the HTTP status and application error code.
- Capture `request_id` when present.
- Check the downstream service used by the route.

Example

For route isolation, compare one known-good case and one failing case. The difference helps establish whether the issue is account data, one resource, configuration, or service availability.

Do not assume

A successful health endpoint or another unrelated route does not prove this route's dependency is healthy.

Request example

A route investigation should capture method, path, status, application error code, `request_id`, and a small response excerpt where safe. Comparing the failing request with one known-good request often shows whether the problem is data-specific or service-wide.

If a route calls several services, record which stage failed. This prevents a browser-level message such as 'order page failed' from hiding the difference between order details, tracking, and documents.

Boundary cases

- Expected 404 should remain a controlled not-found response.
- Validation errors should not be rewritten as server failures.
- Partial tracking should stay marked partial.
- A successful enqueue is not proof of immediate notification delivery.

Endpoint documentation describes expected behavior. Current logs and snapshots are still required to diagnose why a particular request differed from that behavior.